HADI KHAN

Senior Node.js Backend Engineer · 6 Years · Node.js · Express · PostgreSQL · Redis · AWS

Bareilly, UP | +91 9528091316 | khanhadi6@gmail.com | LinkedIn | GitHub | Portfolio

SUMMARY

Six years into backend engineering and the problems that still excite me most are the messy ones — a government API timing out at 6.2 seconds with a million records on the other side, a payment webhook that needs to be idempotent or someone gets charged twice, a PostgreSQL table that EXPLAIN ANALYZE says is doing a full sequential scan on 40 million rows. That's the work I've been doing at HSC and LTIMindtree. Node.js, Express, NestJS, PostgreSQL, Redis, AWS Lambda and SQS are my daily stack. I've used them to cut that 6.2s latency down to 0.5s, wire up a Stripe integration that's run cleanly for four years on a platform doing $75M a year, and pull roughly 40% faster query times out of indexes and partitioning. I also use GitHub Copilot and Claude day-to-day — not as a shortcut, but because writing the test first and having AI fill the boilerplate lets me think about the actual problem.

TECHNICAL SKILLS

Backend	Node.js, Express.js, NestJS, REST APIs, Microservices, event-driven architecture, WebSockets/Socket.IO
Databases	PostgreSQL (indexing, partitioning, EXPLAIN ANALYZE, query tuning), MySQL, Redis (caching, pub/sub, rate limiting)
Cloud & Infra	AWS Lambda, SQS, API Gateway, S3, IAM, Docker, GitHub Actions, CI/CD, Git
Languages	TypeScript, JavaScript (ES6+), SQL — primary; Python (familiar)
Integrations	Stripe (payment intents, webhooks, idempotency, reconciliation), OpenAI API, JWT/OAuth2
Testing	Jest, Supertest, TDD, integration testing, production monitoring
GenAI Tools	GitHub Copilot, Claude (Anthropic), ChatGPT — code generation, debugging, test writing

PROFESSIONAL EXPERIENCE

LTIMindtree · Senior Software Engineer — *Jan 2026 – Present | Remote, India*

- The core challenge was document processing speed. Reports that used to take 45+ minutes to assemble manually now go through an OpenAI API pipeline backed by Node.js/Express microservices and come out structured in under 3 seconds.
- Designed the REST API layer with TypeScript throughout — strict typing across request/response schemas, custom error middleware, and request-level logging that made debugging in production considerably less painful than it could've been.
- Pair-programming with GitHub Copilot and Claude became part of the workflow here. Test scaffolding, edge-case generation, refactoring — it shaved meaningful time off delivery without reducing the quality bar.

HughesSystique Corporation (HSC) · Senior Software Engineer — *Oct 2021 – Oct 2025 | Noida, India*

- The latency problem was the first thing handed to me — a Node.js/Express API serving a government compliance platform, response times at 6.2 seconds, users complaining. Profiling showed N+1 query patterns and missing indexes on join columns. After composite indexes, partial indexes on frequently filtered columns, Redis caching for repeated lookups, and pushing heavy aggregations into async Lambda/SQS jobs, we landed at 0.5 seconds. That's a 12x improvement that held up under production load.
- PostgreSQL tuning was ongoing work, not a one-time fix. EXPLAIN ANALYZE revealed sequential scans on tables with 40M+ rows; table partitioning by date range and query plan rewrites brought retrieval times down roughly 40% across the board.
- Stripe integration was the other major project — payment intents, webhook signature verification, idempotency keys on every charge attempt, daily reconciliation jobs to catch discrepancies. The platform was doing around $75M annually. Four years, zero payment incidents.
- AWS Lambda and SQS handled the async pipelines. Anything that didn't need to block the HTTP response — report generation, email notifications, audit logging — went through there. Meaningful cost reduction and the API stayed responsive under load spikes.
- Code reviews, TDD, and GitHub Actions CI/CD were team standards I helped establish. Production bug rate dropped 15%. Also mentored three junior engineers — weekly 1:1s and structured PR feedback; the team's sprint defect rate came down about 20% over six months.

Svayam Infoware Pvt. Ltd. · Software Engineer — *Jan 2020 – Oct 2021 | Lucknow, India*

- Full-stack ERP built on Node.js/Express and SQL Server, used daily by 200+ employees for inventory, orders, and HR workflows. Most of the backend performance gains came from query rewrites — indexed joins replaced full-table scans, stored procedures handled the reporting load, and average query time dropped roughly 40%.
- Background job processing (Bull + Redis queues) replaced the synchronous report generation and email notification calls that were degrading API response times during peak hours. Operational costs came down around 18% as a result.

KEY ACHIEVEMENTS

12x	Latency cut on a 1M+ record govt platform — 6.2s down to 0.5s via indexing, Redis caching, async SQS pipelines	**$75M**	Annual revenue Stripe integration — idempotency, webhooks, reconciliation, zero incidents in 4 years
~40%	PostgreSQL query time improvement — composite indexes, partitioning, EXPLAIN ANALYZE-guided rewrites	**1M+**	Daily records handled on distributed Node.js microservices with sub-second API response times

EDUCATION

B.Tech, Information Technology · Shri Ramswaroop Memorial College of Engineering, Lucknow · 2020